AGENT EVALUATION · ASSETOPSBENCH · DETERMINISTIC TRUST LAYERS

# Evaluating an Asset-Operations Agent: A Trust-Layer Blueprint on AssetOpsBench

**Abstract.** AssetOpsBench, IBM Research's benchmark for industrial asset-operations agents, poses questions no single tool can answer alone. An agent must retrieve sensor telemetry, identify known failure modes, run a forecasting or anomaly-detection model, and decide whether a work order is warranted. That means coordinating across five MCP tool servers. This report builds a two-layer deterministic trust layer for a small model. The process was iterative: post-hoc checks exposed specific, recurring failure modes in the agent's trajectories. Each one was addressed with a targeted in-loop verifier, then re-measured before moving to the next. This is a hill-climbing process. The result is evaluated against GLM-4.6 and Claude Sonnet 4.5 on the same scenarios. The small model plus trust layer lands within 9 points of a frontier model's success rate, at roughly 1/31st the cost.

August 2026 · gpt-4o-mini, GLM-4.6, and Claude Sonnet 4.5, all via OpenRouter · three evaluation runs: 10 + 16 + 12 scenarios

| | | | |
|---|---|---|---|
| **31×** | **58% / 67%** | **$0.0085** | **3 / 3** |
| Claude Sonnet 4.5's cost vs. gpt-4o-mini, same 12 scenarios, same trust layer | success rate — gpt-4o-mini vs. Claude Sonnet 4.5 | gpt-4o-mini's cost per scenario, vs. $0.263 for Claude Sonnet 4.5 | scenarios both models failed identically — the remaining gap is not purely attributable to capability |

## THE CHALLENGE

Running a small open-weight model against AssetOpsBench surfaced three recurring failure modes. Each one is invisible to a check that asks only whether a tool call succeeded.

- **Structural success, semantic failure.** A trajectory with zero tool errors and fully schema-valid arguments still produced an incorrect answer on a meaningful share of hand-graded runs. Every call succeeded. Some arguments were simply wrong for the asset in question.
- **Silent category skipping.** When a question implied that work orders should be checked, the baseline agent invoked the work-order tool in only 6% of cases (1 of 16 scenarios). It still answered confidently, without making the call a correct answer required.
- **Unproductive retries.** An empty result from a model-catalog lookup was treated as ambiguous rather than definitive. The agent retried the same search with different parameters instead of concluding that no model existed for the task. This consumed turns needed elsewhere in the trajectory.

## THE SOLUTION

Two layers of deterministic logic wrap the same underlying model. Neither is itself an LLM call. **Post-hoc checks** score a completed trajectory. This is useful for reporting, but it happens too late to affect the run being scored. **In-loop verifiers** inspect each tool call as it occurs. When a call appears ungrounded, a verifier injects a corrective observation into the conversation. This is the same mechanism the agent loop already uses to surface tool errors, extended here to answers that are wrong without being invalid. A third mechanism, a pre-finalization gate, prevents the agent from accepting its own answer as final. It blocks until every tool category implied by the question has actually been used.

The two layers were built in that order. The second exists because of the first. Post-hoc checks ran against real trajectories before any in-loop mechanism existed. Each recurring failure they exposed was diagnosed individually: a silently skipped tool category, a wrong argument value, an unproductive retry loop. Each one was then addressed with a narrowly targeted verifier before moving to the next. This is a hill-climbing process. Take the smallest step that reduces an observed failure. Re-measure with the same post-hoc checks. Repeat. The risk is overfitting to exactly the failures already seen. That is why every new check and verifier described below was validated against a scenario set broader than the case that motivated it, not just the one that surfaced the problem.

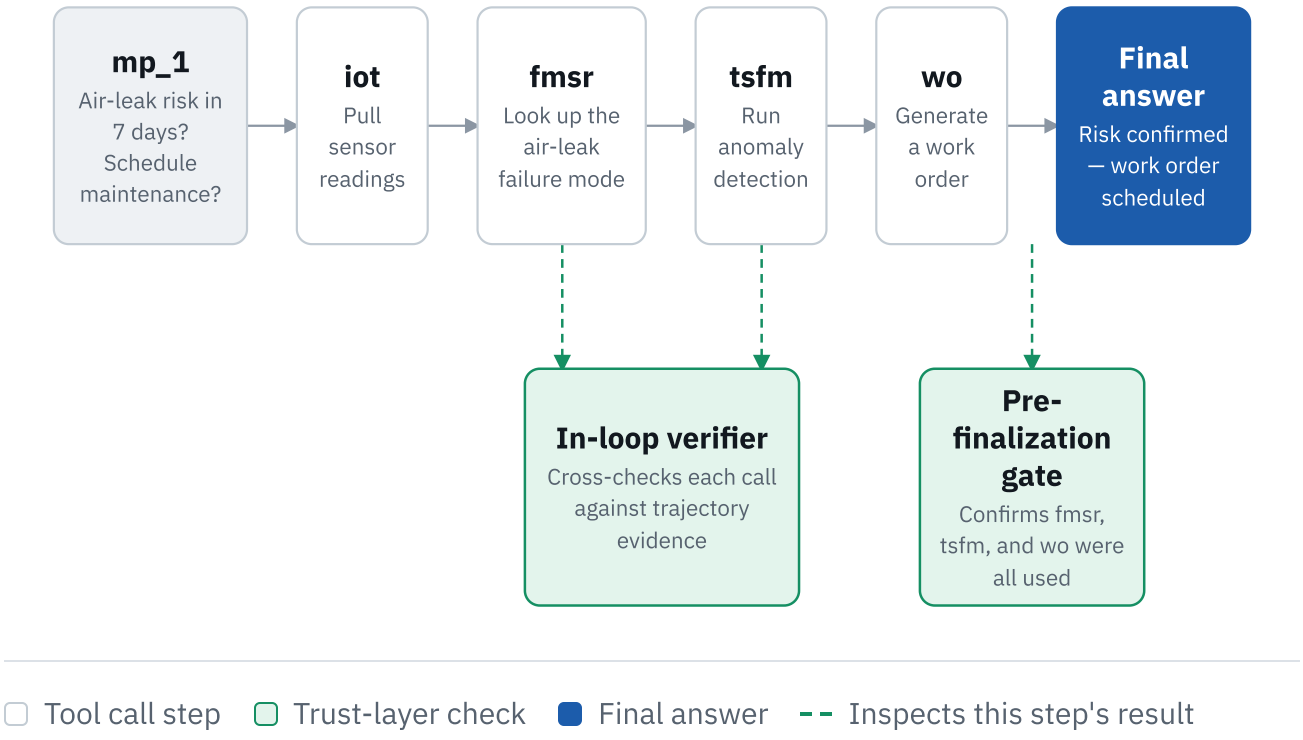
## HOW THE AGENT WORKS

Each scenario runs through a single ReAct-style loop: one model, one shared conversation thread. There is no hand-off between separate per-category agents. The tool catalog is scoped to the servers a given scenario requires. The example below traces one real scenario end to end. The trust layer's two checks are shown as a separate lane beneath the main flow, connected at the points where they fire.

| SERVER | WHAT IT DOES | TOOLS REFERENCED IN THIS STUDY |
|---|---|---|
| **iot** | Asset registry and sensor telemetry — sites, asset metadata, installed/measured sensors, historical readings. | `asset_detail`, `measured_sensors`, `history` |
| **fmsr** | Domain reference for failure modes — known failure modes per asset class and the sensors that can detect them. | `get_failure_modes` |
| **tsfm** | Time-series ML — profiles and cleans sensor data, searches a catalog of pretrained forecasting/anomaly models, runs a recipe to fit and predict. | `find_models`, `run_recipe`, `data_quality` |
| **wo** | Maintenance work orders — list, generate, update, approve, assign, and close; also costs and technician schedules. | `generate_work_order`, `list_workorders` |
| **vibration** | Vibration-signal analysis — raw waveforms, FFT/envelope spectra, severity assessment, bearing fault frequencies, diagnosis. | `get_vibration_data`, `diagnose_vibration` |

**Scenario walkthrough: id 1001**

*"Consider asset mp_1. Based on the compressor sensor readings from April 10 to April 17, 2020, is there a risk of an air leak failure occurring within the next 7 days? Should we schedule maintenance to prevent this issue?"*




## THE TRUST LAYER: TWO KINDS OF DETERMINISTIC CHECKS

Every check described below is implemented as plain code, not a second model call. Each is inexpensive to run and unambiguous to interpret. Each either scores a run after completion, or acts on it while still in progress.

POST-HOC CHECKS: SCORE A COMPLETED TRAJECTORY, NO MODEL CALL INVOLVED

| CHECK | TYPE | WHAT IT VERIFIES |
|---|---|---|
| `no_silent_failures` | STRUCTURAL | Verifies that every tool call in the trajectory returned a non-error result. Flags any call whose output resembles an error payload. This catches a tool that failed silently and was treated as a normal observation — a failure mode that a check verifying only whether a tool was called would miss. |
| `tool_args_match_schema` | STRUCTURAL | Verifies that every tool call's arguments validate against that tool's live JSON schema. Required fields must be present and correctly typed. Schemas are fetched from the MCP servers at the start of each batch, not hand-copied, so the check stays correct if a server's schema changes. |
| `expected_servers_invoked` | STRUCTURAL | For scenarios that declare which tool servers a correct answer requires, verifies that each declared server was invoked at least once. This holds regardless of whether the invoked calls succeeded. |
| `predictive_task_uses_expected_tools` | SEMANTIC | Three regular expressions detect implied intent in the question text: forecast/anomaly language, maintenance-action language, and failure-mode/root-cause language. Each verifies that the corresponding tool category ( `tsfm` , `wo` , `fmsr` ) was invoked. Every pattern was validated against the full diverse scenario set before use. It must fire on scenarios that genuinely ask that kind of question. It must stay silent on |

| CHECK | TYPE | WHAT IT VERIFIES |
| --- | --- | --- |
| | | scenarios that merely mention a named failure type in passing. |
| `answer_shape_matches_characteristic_form` | STRUCTURAL | When a scenario's ground truth declares a JSON object or list shape, verifies that the final answer's shape approximately matches. A question requesting "a list of X," for example, should not be answered with unstructured prose. |

IN-LOOP VERIFIERS: FIRE DURING THE RUN, CAN CHANGE ITS OUTCOME

| VERIFIER | TYPE | WHAT IT DOES |
|---|---|---|
| fmsr asset-class grounding | REACTIVE | Cross-checks the `asset_class` argument passed to `fmsr.get_failure_modes` against the asset's actual `assettype`. It uses any `iot.asset_detail` call for that asset present in the trajectory, regardless of call order — the confirming evidence sometimes arrives after the fmsr call it should ground. A mismatch injects a corrective observation, instructing the model to re-call fmsr with the correct class before finalizing. |
| work-order filter grounding | REACTIVE | Flags a `wo.*` call that narrows results with a filter (`status`, `worktype`, `wopriority`) not requested in the question. This prevents the model from presenting a filtered subset as if it were the complete answer. |
| tsfm empty-catalog grounding | REACTIVE | Fires the first time a tsfm catalog-lookup tool returns zero results. It states explicitly that no model is registered for the task, rather than leaving the result ambiguous. That ambiguity used to drive repeated retries with different parameters, against a search that was already correct. |
| pre-finalization gate | ENFORCED | Applies the same three intent regular expressions as `predictive_task_uses_expected_tools`. The difference is timing: this runs before the model's answer is accepted, not after. If a turn with no tool calls looks like a final answer, and an implied tool category was never invoked, finalization is blocked and a corrective nudge is injected. This happens once per category, so a model with a legitimate reason to omit a tool is not repeatedly renudged for the rest of the run. |

A third, smaller mechanism operates alongside these two. Client-side JSON-schema enrichment adds structure to tool parameters that the servers themselves expose only loosely. This gives the model a concrete shape to populate at the moment it builds a call, instead of requiring it to separately consult prose documentation.

EVALUATION SETUP

Three evaluation runs underlie the results below. Each draws real AssetOpsBench question text — no invented questions. Scenarios were selected in id order within each set, with no cherry-picking.

| STUDY | SCENARIO SET | COUNT | MODEL(S) TESTED |
|---|---|---|---|
| **Structural checks vs. correctness** | Hand-graded scenarios with known ground-truth answers | 10 | gpt-4o-mini |
| **Baseline vs. trust layer** | Predictive-maintenance multiagent scenarios, assets `mp_1` and `hp_1` | 16 | gpt-4o-mini |
| **Cost vs. accuracy** | Diverse sample: IoT-only lookups, fmsr diagnostics, tsfm forecasting/anomaly detection, and full multi-tool `mp_1` / `hp_1` scenarios | 12 | gpt-4o-mini, GLM-4.6, Claude Sonnet 4.5 |

Collectively, these scenarios span three assets: Chiller 6 at the MAIN site, compressor `mp_1`, and hydraulic pump `hp_1` (the real UCI hydraulic-systems sensor set). They ran against an environment seeded with the failure-mode, work-order, and sensor records those scenarios reference. Scenario ids overlap across studies. The `mp_1` / `hp_1` questions used in the cost/accuracy sample are drawn from the same pool as the baseline/trust-layer study. Read the three counts above as the size of each individual run, not as a sum toward a larger unique total.

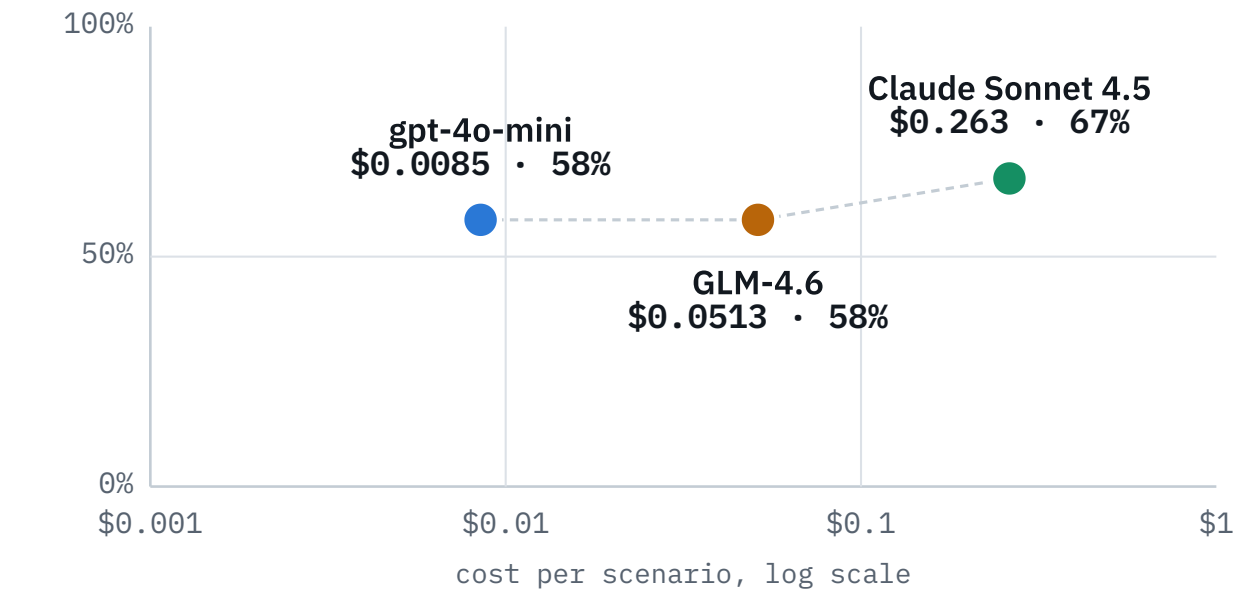
RESULTS AND IMPACT

## Cost vs. accuracy: gpt-4o-mini, GLM-4.6, and Claude Sonnet 4.5

All three models were evaluated on the same 12-scenario sample. It spans IoT-only lookups, failure-mode diagnostics, time-series forecasting/anomaly detection, and full multi-tool predictive-maintenance questions across three assets. All three used the same trust-layer harness — prompt guidance plus enforced verifier gates — and the same deterministic checks. All three were accessed via OpenRouter.

**Cost vs. success rate**

cost per scenario (log scale) vs. success rate, same 12 scenarios



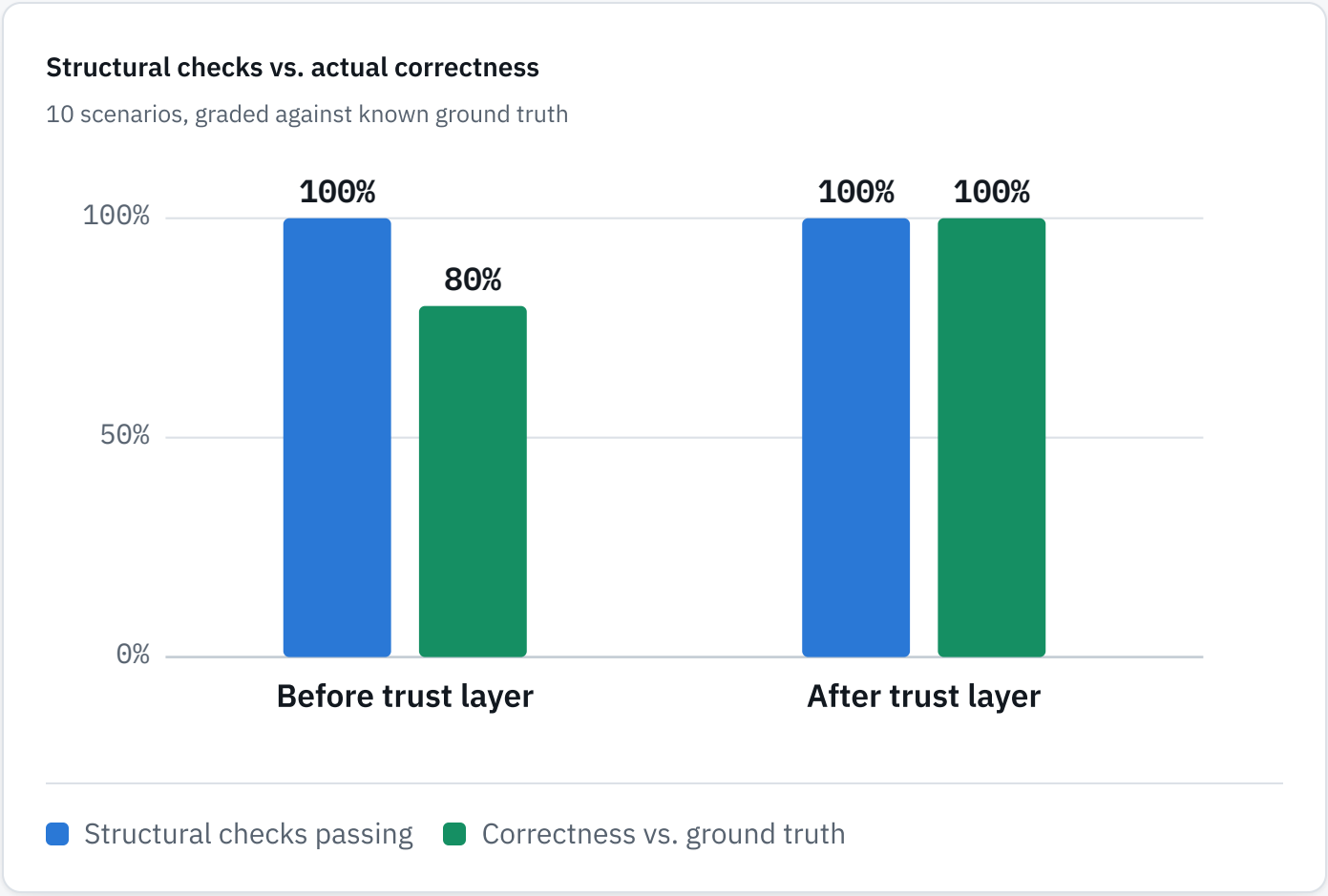

gpt-4o-mini GLM-4.6 Claude Sonnet 4.5

| METRIC | GPT-4O-MINI | GLM-4.6 | CLAUDE SONNET 4.5 |
|---|---|---|---|
| Success rate (0 failed checks) | 58% (7/12) | 58% (7/12) | 67% (8/12) |
| Tokens (in / out) | 649,624 / 7,149 | 1,149,974 / 20,319 | 982,232 / 13,574 |
| Tool calls | 57 | 74 | 68 |
| Total cost, 12 scenarios | $0.10 | $0.62 | $3.15 |
| Cost per scenario | $0.0085 | $0.0513 | $0.263 |

**The gap is not purely attributable to capability.** All three models failed the same three scenarios in the same way. None of them ever invoked the work-order or vibration tools a given question implied — regardless of vendor or price tier. GLM-4.6 matched gpt-4o-mini's success rate exactly, at roughly six times the cost. It also introduced two distinct tool-execution errors that gpt-4o-mini did not hit on this sample. Claude Sonnet 4.5's one distinct failure differed in kind, not degree. It attempted a tool call against a server not registered for that scenario, producing a malformed call. gpt-4o-mini simply did not act on the same prompt cue. Sonnet's advantage on this sample reduces to a single scenario, where it used the work-order tool and the other two models did not.

## Structural checks vs. actual correctness

Ten scenarios with known ground truth were graded by hand. Structural checks — no silent tool failures, every tool argument schema-valid — passed on 100% of runs. Actual correctness against ground truth was **80%**. The in-loop verifiers described above detect semantically incorrect tool arguments, not only malformed ones. Adding them closed this gap, on the same model and scenarios.



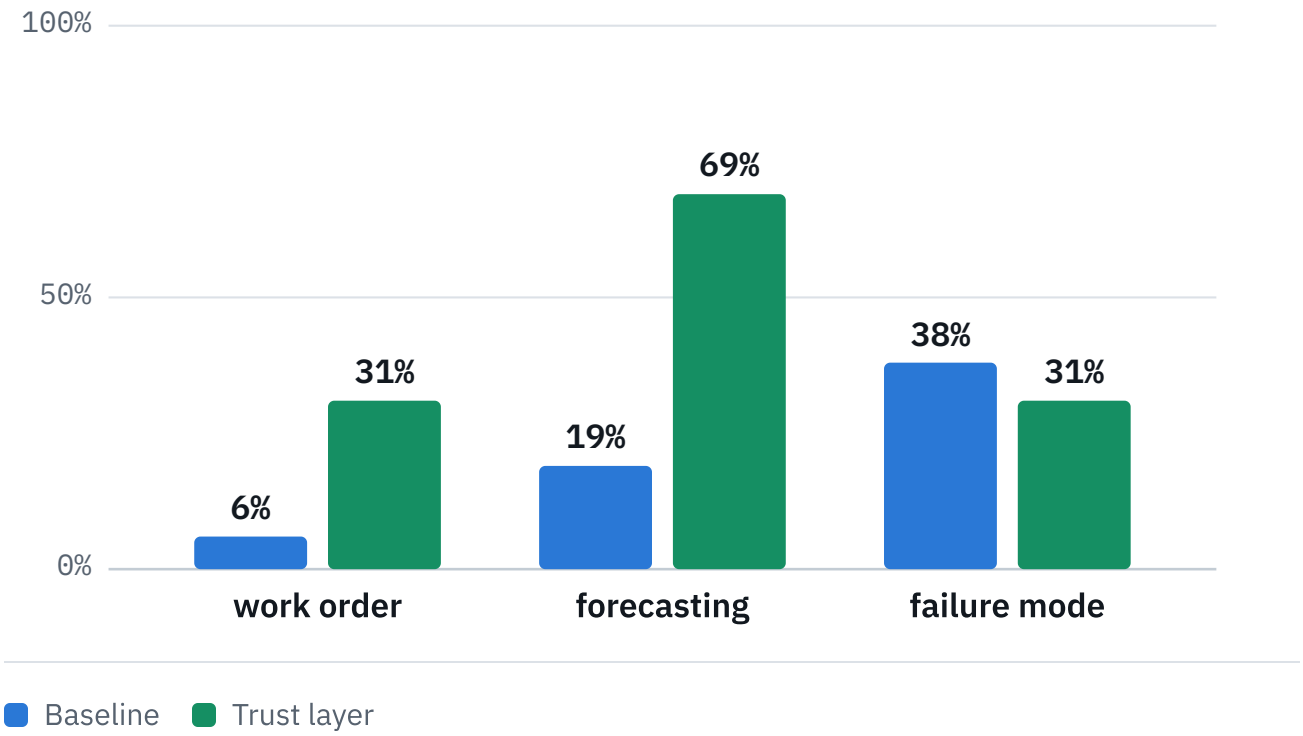
## Baseline vs. trust layer: tool invocation rate

Sixteen predictive-maintenance scenarios were evaluated. The baseline condition is the agent without the trust layer. The trust-layer condition adds the targeted prompt instructions and the enforced pre-finalization gate described above.

**Work orders were the hardest category to close.** Baseline tool use was 6% (1/16) for work orders, versus 19% for forecasting and 38% for failure-mode lookups. The trust layer more than triples work-order invocation, to 31%. That is the largest relative gain of the three categories.

**Tool invocation rate, baseline vs. trust layer**

16 predictive-maintenance scenarios




| METRIC | BASELINE | TRUST LAYER |
|---|---|---|
| Work order tool used | 6% | 31% |
| Forecasting tool used | 19% | 69% |
| Failure-mode tool used | 38% | 31% |
| Full expected agent set (strict) | 6% | 25% |

The hardest scenarios require `fmsr`, `iot`, `tsfm`, and `wo` to cooperate on a single answer. No single category is enough on its own. This is also why a multi-agent split — one agent per tool category — is a harder redesign than it first looks. It would need a router to select the right agents. It would need a supervisor to reconcile their outputs. And it would have to do both without breaking the cross-tool grounding checks described above, which depend on seeing every tool call within one shared trajectory.

## KEY TAKEAWAYS

- → **A small model combined with a deterministic trust layer closes most of the gap to a frontier model, at a fraction of the cost.** Success rates of 58% versus 67% on the same 12 scenarios were achieved at approximately 1/31st the price.
- → **Structural correctness and semantic correctness are distinct measurements.** A trajectory can pass every schema and error check and still be incorrect 20% of the time; only checks that evaluate argument values against trajectory evidence detect this gap.
- → **An enforced gate outperforms guidance alone.** Adding a hard block on finalization to the same prompt instruction moved work-order tool use from 6% to 31%. That more than tripled the hardest category to close.
- → **Most of the remaining gap is concentrated in multi-tool-category coordination, not raw model capability.** All three models failed the same three scenarios in the same way. Each one needs four tool categories to cooperate on a single answer.
- → **This coordination requirement is also why a multi-agent split is not an obvious next step.** The hardest scenarios require shared trajectory context across tool categories. A router-plus-supervisor architecture would need to explicitly reconstruct that context.

## CONCLUSION

The central finding generalizes to any setting where agents are evaluated in production. A fluent answer does not mean the agent performed the correct actions. Passing structural tool-call checks does not mean the arguments inside those calls were correct. A two-layer trust layer closed most of this gap for a small, inexpensive model. Deterministic checks score a run; in-loop verifiers can alter one. Neither required the model itself to grow larger or more expensive.

These results come from three evaluation runs, of 10, 16, and 12 scenarios. This is not an exhaustive benchmark. Read the percentages as directional, not final. Expect additional run-to-run variance from the models themselves. The datasets, checks, and verifiers described above are the reproducible part. The numbers are a snapshot from one run through them.

---

AssetOpsBench (IBM Research) · gpt-4o-mini, GLM-4.6, and Claude Sonnet 4.5 via OpenRouter · August 2026

Code & reproduction steps: github.com/nagarash/assetopsbench-trust-layer